

DEADLOCK GUARDIAN

11 Haftalık El Kitabı ve Bilgi Havuzu

Projeyi yaparken karşılaşacağımız tüm kavramlar, tanımlar,
açıklamalar ve örnekler — hafta hafta

İşletim Sistemleri Dersi · Yazılım Mühendisliği Bölümü

Ana kaynak: Silberschatz, Galvin, Gagne — *Operating System Concepts*, 10. baskı

İçindekiler

0. Başlamadan Önce	3
1. Hafta — Temel Kavramlar ve Deadlock	5
2. Hafta — Basit Simülasyon	9
3. Hafta — Kaynak İlişkileri ve Veri Yapıları	11
4. Hafta — Deadlock Tespiti (Detection)	13
5. Hafta — Görselleştirme	16
6. Hafta — Risk Analizi: Güvenli Durum ve Banker's Algorithm	17
7. Hafta — Risk Seviyesi Sistemi	20
8. Hafta — Recovery (Kurtarma)	21
9. Hafta — Kullanıcı Arayüzü ve Yazılım Mimarisi	23
10. Hafta — Test ve İyileştirme	25
11. Hafta — Final: Demo, Rapor ve Sunum	27
Ek A — Yazılım Mühendisliği Pratikleri	28
Ek B — Python Hızlı Başvuru	29
Ek C — Sözlük	30

0. Başlamadan Önce

Bu kitap nasıl kullanılır, proje nedir, bölümümüzle ne ilgisi var

Bu kitap nasıl kullanılır?

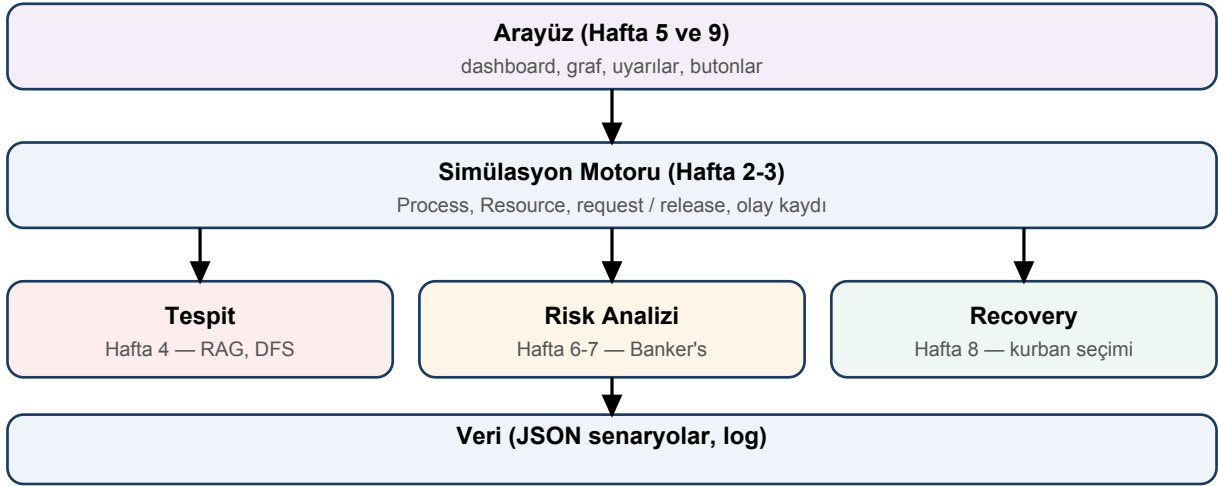
Her bölüm, 11 haftalık plandaki bir haftaya karşılık gelir. O haftaya başlamadan önce ilgili bölümü okuyun. Her bölümde aynı kutu türlerini göreceksiniz:

Kutu	Ne anlatır?
Tanım	Terimin kısa ve kesin anlamı. Sınavda ve raporda kullanılabilecek cümle.
Örnek	Kavramı somutlaştıran gündelik hayat ya da sayısal örnek.
Dikkat	Sık yapılan hatalar ve karıştırılan kavramlar.
Projemizde	Kavramın Deadlock Guardian'da tam olarak nereye karşılık geldiği.
Bilmeniz gerekenler	Bölüm sonu kontrol listesi. Hepsini açıklayabiliyorsanız o haftaya hazırsınız.

Terimler kitapta İngilizce karşılıklarıyla birlikte verilmiştir; çünkü hem ders kitabı hem de kod İngilizcedir. Kitabın sonunda Türkçe-İngilizce bir sözlük vardır (Ek C).

Proje tek paragrafta

Bilgisayarda aynı anda birçok program (process) çalışır ve bu programlar dosya, yazıcı, kilit gibi kaynakları paylaşır. Bazen iki program birbirinin elindeki kaynağı bekler ve ikisi de sonsuza kadar takılı kalır: buna **deadlock** denir. **Deadlock Guardian**, bu durumu kontrollü bir simülasyon ortamında canlandıran bir yazılımdır. Deadlock olmadan önce riski ölçer ve uyarır (**risk analizi**), olduğunda bulur (**tespit**) ve sistemi tekrar çalışır hale getirir (**recovery**).



Şekil 0 — Deadlock Guardian'ın katmanları ve hangi haftada yapıldıkları

Bu proje ve Yazılım Mühendisliği bölümümüz

İşletim Sistemleri dersi, bir yazılımın altında çalışan katmanı öğretir. Yazılım Mühendisliği ise bir yazılımın **planlı, ekipçe, test edilerek ve belgelenerek** nasıl üretileceğini öğretir. Bu proje ikisini birleştirir: konu işletim sistemlerinden, yapılış biçimi yazılım mühendisliğinden gelir. Hoca projeyi değerlendirirken yalnızca algoritmanın doğruluğuna değil, projenin nasıl yönetildiğine de bakacaktır.

Yazılım Mühendisliği kavramı	Bu projede nerede karşımıza çıkıyor?
Yazılım geliştirme yaşam döngüsü (SDLC)	Plan → tasarım → kodlama → test → teslim. 11 haftalık plan bu döngünün ta kendisi.
Artımlı (incremental) geliştirme	Her hafta çalışan sisteme yeni bir parça eklenir; 2. haftanın simülasyonu 4. haftada tespitle, 8. haftada recovery ile büyür.
Gereksinim analizi	Sistemin ne yapması gerektiğinin yazılması (Ek A'da örnekler var).
Modüler tasarım	Simülasyon, tespit, risk, recovery ve arayüz ayrı modüller; 4 kişi aynı anda çalışabilir.
Nesne yönelimli programlama (OOP)	Process ve Resource birer sınıf (class) olarak yazılır.
Sürüm kontrolü (Git)	Dört kişinin kodu GitHub'da birleştirilir; kim neyi ne zaman değiştirdi görülür.
Yazılım testi	10. hafta ve her modülün kendi birim testleri.
Dokümantasyon	Bu el kitabı, Obsidian notları, README ve final raporu.

Projemizde

Hafta 1'in görev dağılımı: **Üye 1** simülasyon ve veri modeli, **Üye 2** tespit ve recovery, **Üye 3** risk analizi, **Üye 4** arayüz ve görselleştirme. Herkes kendi haftalarının bölümünü derinlemesine, diğer bölümleri ise en az bir kez okumalıdır.

1. Hafta — Temel Kavramlar ve Deadlock

Konuyu anlama ve proje tasarımı

1.1 İşletim sistemi nedir?

Tanım

İşletim sistemi (operating system), donanım ile kullanıcı programları arasında duran ve bilgisayarın kaynaklarını (işlemci, bellek, disk, cihazlar) programlar arasında paylaştıran yazılımdır. Ders kitabı onu bir **kaynak dağıtıcı (resource allocator)** olarak tanımlar.

Kaynak dağıtıcı olmanın zor kısmı, kaynakların sınırlı, isteyenlerin ise çok olmasıdır. İşletim sistemi "kime, neyi, ne zaman vereceğine" karar verir. Deadlock da tam bu kararların yanlış bir sıraya girmesiyle ortaya çıkar.

1.2 Program ve process

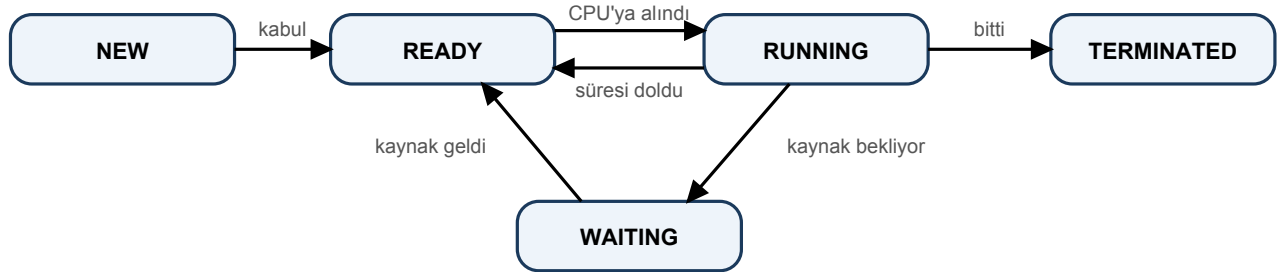
Tanım

Program: diskte duran, çalışmayan komutlar dosyası (ör. *chrome.exe*). Pasiftir.

Process (süreç): çalışmakta olan program. Aktiftir; belleği, durumu ve kullandığı kaynakları vardır. Aynı programı iki kez açarsanız iki ayrı process oluşur.

İşletim sistemi her process için bir **Process Control Block (PCB)** tutar: process'in kimliği (PID), durumu, önceliği ve elindeki kaynaklar bu kayıta durur. Bizim yazacağımız *Process* sınıfı, PCB'nin küçük bir benzeridir.

Process durumları



Şekil 1 — Process durumları ve aralarındaki geçişler (Silberschatz, Bölüm 3)

Durum	Anlamı
NEW	Process oluşturuluyor.
READY	Çalışmaya hazır, işlemci sırası bekliyor.
RUNNING	Şu anda işlemcide çalışıyor.
WAITING	Bir olayı ya da kaynağı bekliyor; o gelmeden ilerleyemez.
TERMINATED	Çalışması bitti.

Projemizde

Simülasyonumuzda işlemci zamanlamasıyla uğraşmadığımız için durumları sadeleştiriyoruz: **RUNNING** (ilerleyebiliyor), **WAITING** (bir kaynağı bekliyor) ve **TERMINATED** (bitti ya da recovery ile sonlandırıldı). Deadlock'taki process'ler hep WAITING durumundadır.

Dikkat

Thread ile process karıştırılmamalı. Thread, bir process'in içindeki çalışma akışıdır; aynı process'in thread'leri belleği paylaşır. Deadlock thread'ler arasında da olur (iki thread iki kilidi ters sırayla almaya çalışırsa). Projemizde "process" diyoruz ama anlattığımız her şey thread'ler için de geçerlidir.

1.3 Kaynak (resource)

Tanım

Kaynak, bir process'in işini yapabilmek için ihtiyaç duyduğu ve sayısı sınırlı olan her şeydir: yazıcı, dosya, veritabanı kaydı, bellek bölgesi, kilit (mutex), ağ bağlantısı...

Kavram	Açıklama	Örnek
Kaynak tipi (resource type)	Aynı türden kaynakların grubu	Yazıcılar = R1
Örnek / adet (instance)	Bir kaynak tipinin kaç tane birbirinin aynısı kopyası olduğu	Laboratuvarda 3 yazıcı → R1'in 3 örneği var
Tek örnekli kaynak	Tipten yalnızca bir tane var	Tek bir dosya kilidi
Çok örnekli kaynak	Tipten birden fazla var; hangisinin verildiği önemsiz	3 yazıcıdan herhangi biri
Paylaşılamaz (non-sharable)	Aynı anda yalnızca bir process kullanabilir	Yazıcı, yazma kilidi
Geri alınamaz (non-preemptable)	Process bırakmadan elinden zorla alınamaz	Yarıda kesilemeyen yazdırma işi

Kaynak kullanım döngüsü

Bir process bir kaynağı her zaman aynı üç adımla kullanır:

- **Request (iste):** kaynak boşsa hemen alınır; doluysa process bekler (WAITING).
- **Use (kullan):** process kaynakla işini yapar.
- **Release (bırak):** process kaynağı serbest bırakır; bekleyen varsa ona verilir.

Projemizde

2. haftada yazacağımız *request()* ve *release()* fonksiyonları bu döngünün koda dökülmüş halidir. "Use" adımını simülasyonda atlarız; kaynağı elinde tutmak kullanıyor sayılır.

1.4 Deadlock nedir?

Tanım

Deadlock (kilitlenme): bir grup process'in her birinin, yine o gruptaki başka bir process'in elindeki kaynağı beklemesi ve bu yüzden hiçbirinin ilerleyememesi durumudur. Dışarıdan bir müdahale olmadan bu bekleme sonsuza kadar sürer.

Örnek

Gündelik hayat: Dar bir köprüye iki yönden aynı anda iki araba girer. İkisi de karşıdakinin geri çekilmesini bekler; kimse geri çekilmezse ikisi de orada kalır.

Bilgisayarda: P1 dosyayı açıp kilitledi, şimdi yazıcıyı istiyor. P2 yazıcıyı almış, şimdi dosyayı istiyor. P1, P2'nin yazıcıyı bırakmasını; P2, P1'in dosyayı bırakmasını bekler. Hiçbiri kendi elindekini bırakmaz.

Process	Elindeki kaynak	Beklediği kaynak
P1	R1 (dosya)	R2 (yazıcı)
P2	R2 (yazıcı)	R1 (dosya)

1.5 Deadlock'un dört koşulu (Coffman koşulları)

Deadlock ancak aşağıdaki **dört koşulun dördü birden** aynı anda sağlanırsa oluşabilir. Biri bile eksikse deadlock olmaz. Bu, konunun en önemli bilgisidir.

Koşul	Anlamı	Yukarıdaki örnekte
1. Mutual exclusion (karşılıklı dışlama)	Kaynak aynı anda yalnızca bir process tarafından kullanılabilir.	Yazıcıyı iki process aynı anda kullanamaz.
2. Hold and wait (tut ve bekle)	Process elindeki kaynağı bırakmadan başka bir kaynak bekler.	P1 dosyayı tutarken yazıcıyı bekliyor.
3. No preemption (geri alınamazlık)	Kaynak, sahibinden zorla alınamaz; sahibi kendisi bırakmalıdır.	Kimse P2'nin elinden yazıcıyı alamıyor.
4. Circular wait (döngüsel bekleme)	$P1 \rightarrow P2 \rightarrow \dots \rightarrow Pn \rightarrow P1$ şeklinde kapalı bir bekleme zinciri vardır.	P1, P2'yi; P2, P1'i bekliyor.

Dikkat

Dört koşul **gerekli** koşullardır, yani deadlock varsa dördü de mutlaka vardır. Ama çok örnekli kaynaklarda dördü birden olsa bile deadlock olmayabilir (Bölüm 4'te göreceğiz). Tek örnekli kaynaklarda ise döngüsel bekleme = deadlock'tur.

1.6 Deadlock ile karıştırılan kavramlar

Kavram	Ne oluyor?	Deadlock'tan farkı
Starvation (açlık)	Bir process kaynağı hiç alamaz, çünkü sıra hep başkalarına gelir.	Sistem ilerliyor; sadece bir process sürekli geride kalıyor.
Livelock	Process'ler sürekli durum değiştirir ama hiçbir işini bitiremez (koridorda sürekli aynı tarafa kaçan iki kişi).	Process'ler WAITING değil, aktif; ama yine de ilerleme yok.
Yavaşlık	Kaynak dolu olduğu için bir süre beklenir, sonra alınır.	Bekleme sonludur; deadlock'ta sonsuzdur.

1.7 Deadlock ile başa çıkmanın dört yolu

Yöntem	Fikir	Projemizde
Prevention (önleme)	Dört koşuldan birini tamamen imkânsız kılmak. En pratiği: kaynakları hep aynı sırayla istemek (döngüsel beklemeyi kırar).	Recovery önerilerinde "kaynak sırasını düzelt" tavsiyesi olarak.
Avoidance (kaçınma)	Her isteği vermeden önce "bu sistemi tehlikeye sokar mı?" diye hesaplamak. Banker's Algorithm.	Hafta 6-7: Risk analizi
Detection + Recovery (tespit ve kurtarma)	Deadlock olmasına izin vermek, sonra bulup çözmek.	Hafta 4 ve Hafta 8
Ignore (görmezden gelme)	Deadlock'un nadir olduğunu varsayıp hiçbir şey yapmamak. Linux ve Windows dahil çoğu işletim sistemi bunu yapar; sorunu uygulama geliştiricisine bırakır.	Bu yüzden projemiz gibi araçlar anlamlıdır.

Projemizde

Deadlock Guardian, ders kitabındaki yöntemlerden üçünü tek sistemde birleştirir: **avoidance** (risk analizi), **detection** (tespit) ve **recovery** (kurtarma). Projeyi özgün yapan da budur.

Bu bölümün sonunda bilmeniz gerekenler

- İşletim sisteminin neden bir kaynak dağıtıcı olduğunu
- Program ile process arasındaki farkı, process durumlarını
- Kaynak tipi ile kaynak örneği (instance) farkını; request → use → release döngüsünü
- Deadlock'un tanımını ve bir örnekle anlatmayı
- Dört Coffman koşulunu ezbere ve örnekleriyle
- Deadlock, starvation ve livelock farkını
- Dört başa çıkma yöntemini ve projemizin hangilerini kullandığını

2. Hafta — Basit Simülasyon

Process ve kaynakları koda dökmek

2.1 Simülasyon nedir, neden gerçek process kullanmıyoruz?

Tanım

Simülasyon, gerçek bir sistemin davranışını yazılım içinde taklit eden modeldir. Bizim simülasyonumuzda "P1" gerçek bir program değil, bilgisayarın belleğinde duran bir nesnedir.

Gerçek işletim sistemindeki process'leri kilitlemek hem tehlikeli (bilgisayar donabilir) hem de kontrolsüzdür (deadlock'u istediğimiz anda oluşturamayız). Simülasyonda ise senaryoyu biz yazarız: "P1 R1'i alsın, P2 R2'yi alsın, sonra çapraz istesinler" deriz ve deadlock tam istediğimiz anda oluşur. Bu karar proje belgesinde de yazılıdır: ilk sürüm kontrollü bir simülasyondur.

2.2 Sınıf ve nesne (OOP)

Tanım

Sınıf (class): bir şeyin kalıbı. "Her process'in bir kimliği ve durumu vardır" der.

Nesne (object): kalıptan üretilmiş somut örnek. P1 ve P2, Process sınıfının iki nesnesidir.

Özellik (attribute): nesnenin taşıdığı veri (pid, state).

Metot (method): nesnenin yapabildiği iş (request, release).

Projede iki temel sınıfımız olacak:

Sınıf	Özellikler	Ne işe yarar?
Process	pid (kimlik), state (durum), held (tuttuğu kaynaklar)	Bir çalışan programı temsil eder.
Resource	rid (kimlik), owner (sahibi), queue (bekleyenler)	Bir kaynağı temsil eder.

2.3 Durum makinesi (state machine)

Process'in durumu rastgele değişmez; yalnızca belirli olaylarla belirli yönlelere geçer. Buna **durum makinesi** denir. Kurallarımız:

Şu anki durum	Olay	Yeni durum
RUNNING	Boş bir kaynağı istedi	RUNNING (kaynak verildi)
RUNNING	Dolu bir kaynağı istedi	WAITING (kuyruğa girdi)
WAITING	Beklediği kaynak kendisine verildi	RUNNING
RUNNING / WAITING	Sonlandırıldı	TERMINATED (tuttuğu her şey bırakılır)

2.4 Kuyruk (queue) ve FIFO

Tanım

Kuyruk, elemanların sona eklenip baştan çıkarıldığı veri yapısıdır. **FIFO** (First In, First Out): ilk gelen ilk çıkar, tıpkı banka sırası gibi.

Bir kaynak doluyken onu isteyen process'ler o kaynağın kuyruğuna girer. Kaynak bırakıldığında kuyruğun başındaki process kaynağı alır. Python'da bunun için `collections.deque` kullanılır: `append()` sona ekler, `popleft()` baştan çıkarır.

Dikkat

FIFO adildir ama deadlock'u engellemez. Kuyruk sırası ne olursa olsun, çapraz bekleme varsa deadlock olur.

2.5 Örnek akış

Adım	Olay	Sonuç	R1'in durumu
1	P1, R1'i istiyor	R1 boş → P1'e verildi	sahibi: P1, kuyruk: boş
2	P2, R1'i istiyor	R1 dolu → P2 WAITING	sahibi: P1, kuyruk: [P2]
3	P1, R1'i bırakıyor	Kuyruğun başı P2 → R1 P2'ye verildi, P2 RUNNING	sahibi: P2, kuyruk: boş

2.6 Kod iskeleti

Aşağıdaki kod 2. haftanın hedefinin en küçük çalışan halidir. Birlikte yazarken bunu adım adım genişleteceğiz; şimdilik mantığı okumanız yeterli.

```
from collections import deque
from enum import Enum

class State(Enum):
    RUNNING = "RUNNING"
    WAITING = "WAITING"
    TERMINATED = "TERMINATED"

class Process:
    def __init__(self, pid):
        self.pid = pid # "P1" gibi kimlik
        self.state = State.RUNNING
        self.held = [] # tuttuğu kaynaklar

class Resource:
    def __init__(self, rid):
        self.rid = rid # "R1" gibi kimlik
        self.owner = None # kaynağı tutan process (yoksa None)
        self.queue = deque() # bekleyen process'ler (FIFO)

def request(p, r):
    if r.owner is None: # kaynak boş → ver
        r.owner = p
        p.held.append(r)
        p.state = State.RUNNING
    else: # kaynak dolu → beklet
        p.state = State.WAITING
        r.queue.append(p)

def release(p, r):
    p.held.remove(r)
    r.owner = None
    if r.queue: # bekleyen varsa sıradakine ver
        request(r.queue.popleft(), r)
```

Bu bölümün sonunda bilmeniz gerekenler

- Neden gerçek process yerine simülasyon kullandığımızı
- Sınıf, nesne, özellik ve metot kavramlarını
- Process durum geçişlerini (durum makinesi)
- Kuyruk ve FIFO mantığını
- request ve release fonksiyonlarının adım adım ne yaptığını

3. Hafta — Kaynak İlişkileri ve Veri Yapıları

Sistemin tam fotoğrafını tutmak

3.1 Neden tablolara ihtiyacımız var?

2. haftada her kaynak yalnızca "sahibim kim?" bilgisini tutuyordu. Ama tespit ve risk analizi algoritmaları sistemin **tamamına** bir bakışta bakmak ister: kim neyi tutuyor, kim neyi istiyor, ne kadar boş kaynak kaldı? Ders kitabı bunun için üç standart yapı kullanır. Bu isimler 4. ve 6. haftada algoritmaların içinde aynen geçecek.

Yapı	Tür	Anlamı
Available	Vektör (her kaynak tipi için bir sayı)	Şu an boşta kaç adet kaynak var?
Allocation	Matris (satır = process, sütun = kaynak tipi)	Her process her kaynaktan kaç adet tutuyor?
Request	Matris	Her process her kaynaktan şu an kaç adet daha istiyor (bekliyor)?
Max (Hafta 6)	Matris	Her process her kaynaktan en fazla kaç adete ihtiyaç duyacağını önceden bildirir.
Need (Hafta 6)	Matris	Need = Max – Allocation: daha ne kadar isteyebilir?

Tanım

Vektör: tek satırlık sayı listesi. [3, 2] → R1'den 3, R2'den 2.

Matris: satır ve sütunlardan oluşan sayı tablosu.

3.2 Çok örnekli kaynaklar: sayısal örnek

Sistemde iki kaynak tipi olsun: R1 = yazıcı (toplam 3 adet), R2 = tarayıcı (toplam 2 adet).

Process	Allocation (R1, R2)	Request (R1, R2)	Açıklama
P1	1, 0	0, 0	1 yazıcı tutuyor, bir şey beklemiyor
P2	1, 1	1, 0	1 yazıcı + 1 tarayıcı tutuyor, 1 yazıcı daha istiyor
P3	1, 1	0, 1	1 yazıcı + 1 tarayıcı tutuyor, 1 tarayıcı daha istiyor

Toplam tahsis edilen: R1 için $1+1+1 = 3$, R2 için $0+1+1 = 2$. Demek ki **Available = [0, 0]**; boşta hiç kaynak yok. Kural her zaman şudur:

Available = Toplam – (tüm process'lerin Allocation satırlarının toplamı)

Dikkat

Tablolar her request ve release'den sonra tutarlı güncellenmelidir. Bir tablodaki hata, tespit algoritmasının yanlış sonuç vermesine yol açar. Bu yüzden 3. haftada her işlemten sonra "toplamlar tutuyor mu?" kontrolü yazmak iyi bir fikirdir.

3.3 Olay kaydı (log)

Tanım

Log, sistemde olan her olayın zaman sırasıyla yazıldığı kayıttır.

```
[0001] P1 REQUEST R1 -> GRANTED
[0002] P2 REQUEST R1 -> WAITING (sahibi P1)
[0003] P1 RELEASE R1 -> R1 P2'ye verildi
```

Log, hata ayıklarken "ne oldu da buraya geldik?" sorusunun cevabıdır. Arayüzde olay akışı olarak da gösterilecek. Zaman damgası olarak gerçek saat yerine adım numarası (tick) kullanmak, testleri tekrar edilebilir yapar.

3.4 JSON ile senaryo dosyası

Tanım

JSON, verileri anahtar-değer çiftleriyle yazan, hem insanın hem programın kolayca okuyabildiği bir metin formatıdır. Python'da *json.load()* ile tek satırda okunur.

```
{
  "resources": {"R1": 1, "R2": 1},
  "processes": ["P1", "P2"],
  "steps": [
    ["P1", "request", "R1"],
    ["P2", "request", "R2"],
    ["P1", "request", "R2"],
    ["P2", "request", "R1"]
  ]
}
```

Bu dosya çalıştırıldığında son iki adımda deadlock oluşur. Test ve demo senaryolarını böyle dosyalarda tutmak, kodu değiştirmeden farklı durumları denemeyi sağlar.

Projemizde

10. haftadaki dört test senaryosu (normal, tek deadlock, çoklu deadlock, recovery) ve final demosu birer JSON dosyası olacak.

Bu bölümün sonunda bilmeniz gerekenler

- Available, Allocation ve Request yapılarının anlamını
- Available'ın nasıl hesaplandığını
- Çok örnekli kaynak tablosunu okuyup yorumlamayı
- Log'un ve senaryo dosyasının neden gerekli olduğunu, JSON formatını

4. Hafta — Deadlock Tespiti (Detection)

Oluşan deadlock'u otomatik bulmak

4.1 Graf nedir?

Tanım

Graf (graph): düğümlerden ve onları bağlayan kenarlardan oluşan yapı.

Düğüm (node / vertex): bir nesne; bizde process ya da kaynak.

Kenar (edge): iki düğüm arasındaki ilişki.

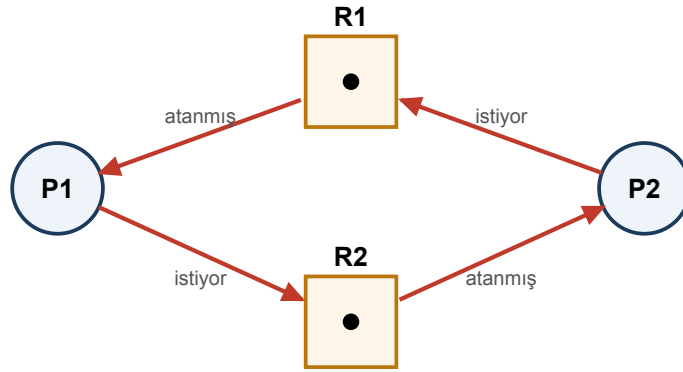
Yönlü graf (directed graph): kenarların oklu olduğu, yönün önemli olduğu graf.

Döngü (cycle): bir düğümden çıkıp okları takip ederek aynı düğüme geri dönebilmek.

4.2 Resource Allocation Graph (RAG)

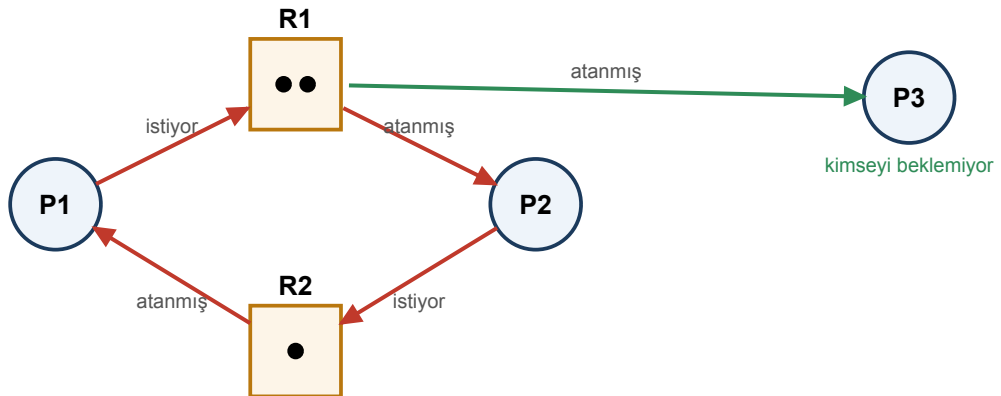
Sistemin durumunu gösteren yönlü graftır. Process'ler **daire**, kaynaklar **kare** ile çizilir; karenin içindeki her nokta bir örnektir (instance).

Kenar	Yön	Anlamı
Request edge (istek kenarı)	Process → Kaynak	Process bu kaynağı istiyor ve bekliyor.
Assignment edge (atama kenarı)	Kaynak → Process	Kaynağın bir örneği bu process'e verilmiş.



Şekil 2 — Deadlock: $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$ döngüsü (her kaynaktan 1 adet var)

Şekil 2'de okları takip edin: P1'den R2'ye, R2'den P2'ye, P2'den R1'e, R1'den tekrar P1'e. Başladığımız yere döndük: **döngü var**. Kaynaklar tek örnekli olduğu için bu kesin bir deadlock'tur.



Şekil 3 — Döngü var ama deadlock yok: R1'in 2 adedi var, P3 bitince bir adet boşalır

Şekil 3'te de $P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1$ döngüsü var. Ama R1'in **iki** örneği var ve biri P3'te. P3 kimseyi beklemediği için işini bitirip R1'i bırakacak. O örnek P1'e verilecek, P1 bitince R2 boşalacak ve P2 de devam

edecek. **Döngü var ama deadlock yok.**

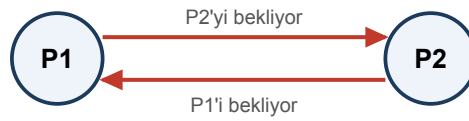
Dikkat

Altın kural:

- Grafta döngü yoksa → kesinlikle deadlock yok.
- Döngü var ve tüm kaynaklar tek örnekliyse → kesinlikle deadlock var.
- Döngü var ama kaynaklar çok örnekliyse → deadlock **olabilir de olmayabilir de**; aşağıdaki detection algoritması ile kontrol etmek gerekir.

4.3 Wait-for graph

Tek örnekli kaynaklarda graf sadeleştirilebilir: kaynak düğümleri silinir, "P1, R2'yi bekliyor ve R2'nin sahibi P2" bilgisi doğrudan **P1 → P2** (P1, P2'yi bekliyor) okuna dönüşür.



Şekil 4 — Şekil 2'nin wait-for graph hali: kaynak düğümleri kaldırılır

4.4 DFS ile döngü bulma

Tanım

DFS (Depth-First Search, derinlik öncelikli arama): bir düğümden başlayıp bir yol boyunca gidebildiği kadar derine giden, çıkmaz sokağa girince geri dönüp başka yolu deneyen graf gezme algoritmasıdır.

Döngü bulmak için düğümleri üç renkle işaretleriz:

Renk	Anlamı
Beyaz	Henüz hiç ziyaret edilmedi.
Gri	Şu an üzerinde yürüdüğümüz yolda (henüz işi bitmedi).
Siyah	Tamamen incelendi; buradan döngüye çıkılmıyor.

Yürürken **gri** bir düğüme tekrar ulaşırsak, kendi yolumuzun bir önceki noktasına geri dönmüşüz demektir: döngü bulundu.

```
def has_cycle(graph):
    """graph: {"P1": ["R2"], "R2": ["P2"], ...} (her düğüm anahtar olmalı)"""
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {node: WHITE for node in graph}

    def dfs(u):
        color[u] = GRAY
        for v in graph[u]:
            if color[v] == GRAY:
                # yolumuzdaki düğüme döndük → döngü
                return True
            if color[v] == WHITE and dfs(v):
                return True
        color[u] = BLACK
        return False

    return any(color[n] == WHITE and dfs(n) for n in graph)
```

4.5 Çok örnekli kaynaklar için detection algoritması

Ders kitabındaki bu algoritma şunu sorar: "Şu an boşta olan kaynaklarla, isteği karşılanabilecek bir process var mı? Varsa onu bitmiş say, kaynaklarını geri al ve tekrar sor." Sonunda bitmiş sayılamayan process'ler deadlock'tadır.

```
Work    = Available          (boştaki kaynakların kopyası)
Finish  = her process için False  (hiç kaynak tutmuyorsa True)

tekrarla:
    Finish[i] == False  VE  Request[i] <= Work   olan bir i bul
    bulunduysa:  Work = Work + Allocation[i]      (i bitti, kaynaklarını bıraktı)
                Finish[i] = True
    bulunamadıysa: dur

Finish[i] == False kalan her process deadlock'tadır.
```

Örnek

Örnek 1 (deadlock yok): 3.2'deki tablo, Available = [0, 0].

- P1'in isteği $[0, 0] \leq [0, 0] \rightarrow$ P1 biter, $Work = [0,0] + [1,0] = [1, 0]$
- P2'nin isteği $[1, 0] \leq [1, 0] \rightarrow$ P2 biter, $Work = [1,0] + [1,1] = [2, 1]$
- P3'ün isteği $[0, 1] \leq [2, 1] \rightarrow$ P3 biter. Herkes bitti: **deadlock yok** (sıra: P1, P2, P3).

Örnek 2 (deadlock var): Aynı tablo, ama P1 de bir tarayıcı istiyor: $Request(P1) = [0, 1]$.

- $Work = [0, 0]$. P1 $[0,1]$, P2 $[1,0]$, P3 $[0,1]$ isteklerinden hiçbiri karşılanamaz.
- Hiçbiri bitemez: **P1, P2 ve P3 deadlock'ta.**

Dikkat

" $[1, 0] \leq [2, 1]$ " ifadesi vektörlerin **her elemanı ayrı ayrı** küçük eşit demektir: $1 \leq 2$ ve $0 \leq 1$. Bir eleman bile büyükse karşılaştırma yanlışır.

4.6 Tespit ne zaman çalıştırılır?

Gerçek sistemlerde tespit pahalı olduğu için belirli aralıklarla çalıştırılır. Bizim simülasyonumuz küçük olduğu için **her request işleminden sonra** çalıştırabiliriz; böylece deadlock olduğu an yakalanır.

Tanım

Birim testi (unit test): kodun tek bir parçasını (ör. *has_cycle* fonksiyonu) tek başına, bilinen bir girdiyle çalıştırıp beklenen çıktıyı verip vermediğini kontrol eden küçük programdır. Ayrıntısı Bölüm 10'da.

Bu bölümün sonunda bilmeniz gerekenler

- Graf, düğüm, kenar, yönlü graf ve döngü kavramlarını
- RAG'de request ve assignment kenarlarını, çizim kurallarını
- Döngü ile deadlock arasındaki ilişkiyi (altın kural) ve Şekil 3'teki istisnayı
- Wait-for graph'ı
- DFS'in üç renkle döngüyü nasıl bulduğunu
- Detection algoritmasını elle bir tablo üzerinde çalıştırabilmeyi

5. Hafta — Görselleştirme

İlişkileri bir bakışta anlaşılır göstermek

5.1 Neden görselleştirme?

Tablolar doğrudur ama okunması zordur. Aynı bilgi RAG olarak çizildiğinde döngü göze hemen çarpar. Projemizi bir algoritma ödevinden ayıran şeylerden biri de, kullanıcının neler olup bittiğini görebilmesidir.

5.2 Çizim kuralları

Öge	Gösterim
Process	Daire; içinde adı (P1)
Kaynak	Kare; üstünde adı (R1), içinde her örnek için bir nokta
Atama kenarı	Kaynaktan process'e ok
İstek kenarı	Process'ten kaynağa ok (tercihen kesikli çizgi)
Deadlock döngüsü	Kırmızı; döngüdeki düğümler vurgulanır
Process durumu	Renk kodu: RUNNING yeşil, WAITING turuncu, TERMINATED gri

5.3 Graf yerleşimi (layout)

Tanım

Layout, düğümlerin ekranda nereye konacağına karar veren yöntemdir. Elle koordinat vermek yerine kütüphaneler bunu otomatik yapar; örneğin *force-directed* yerleşim, düğümleri birbirini iten mıknatıslar, kenarları yaylar gibi düşünerek dengeli bir görüntü üretir.

5.4 Kullanılabilecek araçlar

Araç	Ne işe yarar?	Hangi arayüzle?
networkx	Python'da graf veri yapısı ve algoritmaları; döngü bulma dahil	Her ikisi
matplotlib	Grafı resim olarak çizme	PyQt içine gömülebilir
PyQt QGraphicsView	Etkileşimli, tıklanabilir çizim alanı	PyQt
vis-network / Cytoscape.js	Tarayıcıda etkileşimli graf	Web

Dikkat

networkx'in hazır döngü fonksiyonları var, ama projenin değerlendirilen kısmı algoritmayı **bizim** yazmamız. Hazır fonksiyonları yalnızca kendi kodumuzun sonucunu test etmek için karşılaştırma amacıyla kullanmak doğru olur.

Projemizde

Arayüz teknolojisi (PyQt mı web mi) henüz seçilmedi; Bölüm 9'daki karşılaştırmaya bakarak 5. haftadan önce karar vermemiz gerekiyor, çünkü çizim kodu buna göre yazılacak.

Bu bölümün sonunda bilmeniz gerekenler

- RAG'in ekranda nasıl çizileceğini (şekil ve renk kuralları)
- Layout kavramını
- Hangi araçların ne işe yaradığını ve hazır algoritma kullanmanın sınırını

6. Hafta — Risk Analizi: Güvenli Durum ve Banker's Algorithm

Deadlock oluşmadan önce tehlikeyi görmek (avoidance)

6.1 Avoidance fikri

Tespit (Hafta 4), deadlock **olduktan sonra** çalışır. Avoidance ise her kaynak isteğinde "bu isteği verirsem sistem ileride kilitlenebilir mi?" diye önceden hesaplar. Tehlikeliyse isteği o an vermez, process biraz bekler.

6.2 Güvenli durum (safe state)

Tanım

Güvenli durum: Process'leri bir sıraya dizip, her birinin en kötü ihtimalle ihtiyacının tamamını alarak bitebileceği bir sıra varsa, sistem güvenli durumdadır.

Güvenli sıra (safe sequence): bu bitirme sırasının kendisidir (ör. $\langle P1, P2, P3 \rangle$).

Güvensiz durum (unsafe state): böyle bir sıra yoksa.

Dikkat

Güvensiz durum, deadlock **demek değildir**. Güvensiz durum "deadlock olma ihtimali var" demektir. İlişki şöyledir: tüm deadlock'lar güvensizdir ama tüm güvensiz durumlar deadlock değildir. Güvenli durumdaysak ise deadlock kesinlikle olmaz.

Örnek

Banka benzetmesi (algoritmanın adı buradan gelir): Bankacının 10 lirası var ve üç müşteriye kredi limiti tanımış. Bankacı, bir müşteriye para verirken "bu parayı verdikten sonra, en azından bir müşterinin limitinin tamamını karşılayıp parasını geri alabilir miyim, sonra ondan gelenle diğerini..." diye düşünür. Herkesi sırayla tamamlayabileceği bir yol varsa parayı verir.

6.3 Gerekli yapılar

Banker's Algorithm, 3. haftadaki yapılara iki tane daha ekler:

```
Max[i] = process i'nin her kaynaktan en fazla kaç adete ihtiyacı olacağı (baştan bildirir)
Need[i] = Max[i] - Allocation[i] (daha ne kadar isteyebilir)
```

Dikkat

Algoritmanın en önemli şartı, her process'in **maksimum ihtiyacını önceden bildirmesidir**. Gerçek hayatta bu çoğu zaman bilinmez; algoritmanın pratikteki zayıflığı budur. Simülasyonumuzda bunu senaryo dosyasında her process için yazacağız.

6.4 Safety (güvenlik) algoritması

```
Work = Available
Finish = her process için False
```

tekrarla:

```
    Finish[i] == False VE Need[i] <= Work olan bir i bul
    bulundaysa: Work = Work + Allocation[i]
                Finish[i] = True, i'yi güvenli sıraya ekle
    bulunamadıysa: dur
```

Herkes True ise → GÜVENLİ (bulunan sıra güvenli sıradır), değilse → GÜVENSİZ

Dikkat

Detection algoritması ile neredeyse aynıdır. Tek fark: detection **Request** ile (şu an istediği), safety **Need** ile (en kötü ihtimalle isteyebileceği) karşılaştırır. Bu yüzden iki algoritma aynı yardımcı fonksiyonu paylaşabilir.

6.5 Çalışılmış örnek

Kaynaklar: R1 toplam 6, R2 toplam 4.

Process	Max (R1, R2)	Allocation	Need = Max – Allocation
P1	4, 2	2, 1	2, 1
P2	3, 2	1, 1	2, 1
P3	4, 3	1, 1	3, 2

Tahsis edilen toplam: [4, 3] → **Available** = [6-4, 4-3] = [2, 1].

Örnek

Safety kontrolü: Work = [2, 1]

- P1: Need [2, 1] ≤ [2, 1] (uygun) → Work = [2, 1] + [2, 1] = [4, 2]
- P2: Need [2, 1] ≤ [4, 2] (uygun) → Work = [4, 2] + [1, 1] = [5, 3]
- P3: Need [3, 2] ≤ [5, 3] (uygun) → Work = [5, 3] + [1, 1] = [6, 4]

Sonuç: **GÜVENLİ**, güvenli sıra <P1, P2, P3>.

6.6 Resource-request algoritması

Bir process kaynak istediğinde üç adım uygulanır:

- **Adım 1:** İstek, bildirdiği ihtiyaçtan büyükse (Request > Need) → hata; process yalan söylüyor.
- **Adım 2:** İstek, boştaki kaynaktan büyükse (Request > Available) → process beklemeli.
- **Adım 3:** Kaynağı **verilmiş gibi** yap (Available -, Allocation +, Need -) ve safety algoritmasını çalıştır. Güvenliyse ver; güvensizse eski haline döndür ve process'i beklet.

Örnek

P3, [1, 0] istiyor: [1,0] ≤ Need [3,2] (uygun), [1,0] ≤ Available [2,1] (uygun).

Verilmiş gibi yap: Available = [1, 1], Allocation(P3) = [2, 1], Need(P3) = [2, 2].

Safety: Work = [1, 1]. P1 Need [2,1] (uygun değil), P2 [2,1] (uygun değil), P3 [2,2] (uygun değil) → kimse bitemiyor.

Sonuç: **GÜVENSİZ** → istek reddedilir, P3 bekler.

P1, [1, 0] istiyor: Available = [1, 1], Allocation(P1) = [3, 1], Need(P1) = [1, 1].

Safety: P1 [1,1] ≤ [1,1] (uygun) → Work = [4, 2]; P2 (uygun) → [5, 3]; P3 (uygun) → [6, 4].

Sonuç: **GÜVENLİ** → istek verilir.

Projemizde

Banker's'ı katı bir kapı gibi kullanmak yerine **uyarı** aracı olarak kullanacağız: güvensiz bir istek geldiğinde kullanıcıya "bu istek sistemi güvensiz duruma sokuyor" diyeceğiz. Böylece demoda kullanıcı uyarıyı görmezden gelip deadlock'un oluşmasını da izleyebilecek.

Bu bölümün sonunda bilmeniz gerekenler

- Avoidance ile detection farkını
- Güvenli durum, güvenli sıra ve güvensiz durum tanımlarını; güvensiz $\neq$ deadlock
- Max ve Need yapılarını, Need'in hesabını
- Safety algoritmasını elle çalıştırabilmeyi
- Resource-request algoritmasının üç adımını
- Banker's'ın pratikteki sınırlılığını (maksimum ihtiyacın önceden bilinmesi)

7. Hafta — Risk Seviyesi Sistemi

Riski derecelendirmek ve nedenini açıklamak

Bu bölüm ders kitabında hazır bulunmaz; **projemizin kendi tasarımıdır** ve özgünlüğümüzün önemli bir kısmı buradadır. Aşağıdaki kurallar bir başlangıç önerisidir; eşik değerlerine 7. haftada ekip olarak birlikte karar vereceğiz.

7.1 Neden seviyelere ihtiyaç var?

Banker's Algorithm yalnızca iki cevap verir: güvenli ya da güvensiz. Kullanıcı için daha faydalı olan, "ne kadar tehlikedeyiz ve neden?" sorusunun cevabıdır. Hava durumundaki sarı-turuncu-kırmızı uyarılar gibi düşünün.

7.2 Önerilen seviyeler

Seviye	Koşul (öneri)	Kullanıcıya mesaj örneği
LOW	Bekleyen process yok ya da boş kaynak bol; sistem güvenli.	Sistem normal çalışıyor.
MEDIUM	Sistem güvenli ama bekleyen process var ve boş kaynak az (ör. %20'nin altı) ya da en az iki halkalı bir bekleme zinciri oluşmuş.	P2, P1'i bekliyor; boş kaynak azaldı.
HIGH	Sistem güvensiz durumda (Banker's güvenli sıra bulamıyor) ama henüz deadlock yok.	R2 isteği sistemi güvensiz duruma soktu. P1 ve P3 birbirini bekleyebilir.
CRITICAL	Detection algoritması deadlock buldu.	DEADLOCK: P1 → R2 → P2 → R1 → P1

7.3 Risk nedenini açıklamak

Her uyarı, nedenini söylemelidir. Bunun için risk hesaplanırken hangi kuralın tetiklendiği de kaydedilir. Proje belgesindeki örnek mesaj: "P2 ve P4 karşılıklı kaynak bekliyor."

Tanım

Heuristic (sezgisel kural): kesin matematiksel garanti vermeyen ama pratikte iyi sonuç veren kural. Risk seviyelerimiz bir heuristic'tir; raporda bunu açıkça belirtmeliyiz.

7.4 Zaman içindeki değişim

Risk seviyesi her adımdan sonra kaydedilirse, demoda "LOW → MEDIUM → HIGH → CRITICAL → (recovery) → LOW" geçişi bir zaman çizelgesinde gösterilebilir. Final demo senaryosundaki "sistem risk artışı gösteriyor" adımı tam olarak budur.

Bu bölümün sonunda bilmeniz gerekenler

- Risk seviyelerinin neden Banker's'ın tek başına veremediği bilgiyi sunduğunu
- Dört seviyenin önerilen koşullarını
- Her uyarının bir neden içermesi gerektiğini
- Heuristic kavramını ve kuralların ekip kararı olduğunu

8. Hafta — Recovery (Kurtarma)

Deadlock'tan sonra sistemi tekrar çalışır hale getirmek

Deadlock bir kez oluştuysa, dört koşuldan birini zorla kırmak gerekir. Ders kitabı iki yol önerir: process'leri sonlandırmak ya da kaynakları geri almak.

8.1 Yöntem 1: Process sonlandırma (process termination)

Yaklaşım	Nasıl?	Artısı / eksisi
Hepsini sonlandır	Deadlock'taki tüm process'ler sonlandırılır.	Kesin ve hızlı çözer; ama yapılan tüm iş kaybolur.
Birer birer sonlandır	Bir process sonlandırılır, tespit tekrar çalıştırılır; deadlock sürüyorsa bir sonraki...	Daha az iş kaybı; ama her seferinde tespit tekrar çalışır.

8.2 Kurban seçimi (victim selection)

Tanım

Kurban (victim): deadlock'u çözmek için sonlandırılacak ya da kaynağı elinden alınacak process. Amaç, **en düşük maliyetli** kurbanı seçmektir.

Ders kitabının maliyet ölçütleri:

- Process'in önceliği (önemli process'ler korunur)
- Ne kadar süredir çalıştığı ve bitmesine ne kadar kaldığı
- Kaç kaynak tuttuğu (çok kaynak tutan, sonlanınca çok kaynak serbest bırakır)
- Bitirmek için kaç kaynağa daha ihtiyacı olduğu
- Deadlock'u çözmek için kaç process'in sonlandırılması gerekeceği

Örnek

Simülasyon için basit bir maliyet formülü önerisi:

$$\text{maliyet} = \text{öncelik} \times 10 + \text{tuttuğu kaynak sayısı} \times (-2) + \text{daha önce kurban seçilme sayısı} \times 5$$

Döngüdeki process'lerin maliyeti hesaplanır, en düşük olan sonlandırılır. Formüldeki katsayılar örnektir; 8. haftada birlikte belirleyeceğiz.

8.3 Yöntem 2: Kaynak geri alma (resource preemption)

Process'i tamamen öldürmek yerine elindeki bir kaynağı alıp başkasına vermek. Üç sorunu vardır:

Sorun	Açıklama
Kurban seçimi	Kimden, hangi kaynağı alacağız? (yukarıdaki maliyet mantığı)
Rollback (geri sarma)	Kaynağı alınan process eski bir güvenli noktaya geri döndürülmelidir; en basiti baştan başlatmaktır.
Starvation	Hep aynı process kurban seçilirse hiç bitemez. Çözüm: kurban seçilme sayısını maliyete eklemek.

8.4 Kurtarmadan sonra doğrulama

Kurtarma uygulandıktan sonra detection algoritması **tekrar çalıştırılır**. Deadlock kalmadıysa sistem SAFE durumuna döndü demektir ve risk seviyesi düşer. Bu doğrulama adımı, final demonun son adımıdır.

Projemizde

Arayüzde deadlock oluştuğunda kullanıcıya seçenekler sunulacak: "P2'yi sonlandır (maliyet: 12)", "P1'i sonlandır (maliyet: 18)", "R1'i P2'den geri al". En düşük maliyetli seçenek önerilen olarak işaretlenecek.

Bu bölümün sonunda bilmeniz gerekenler

- İki kurtarma yöntemini ve farklarını
- Hepsini sonlandırma ile birer birer sonlandırma farkını
- Kurban seçim ölçütlerini ve maliyet mantığını
- Rollback ve starvation sorunlarını
- Kurtarmadan sonra neden tespitin tekrar çalıştırıldığını

9. Hafta — Kullanıcı Arayüzü ve Yazılım Mimarisi

Tüm modülleri tek ekranda birleştirmek

9.1 Katmanlı mimari ve sorumlulukların ayrılması

Tanım

Separation of concerns (sorumlulukların ayrılması): her modülün yalnızca kendi işini yapması. Simülasyon motoru ekranı bilmez; arayüz algoritmaları bilmez, sadece motora sorar.

Bu ayırım bize üç şey kazandırır: (1) dört kişi aynı anda çalışabilir, (2) algoritmalar arayüz olmadan test edilebilir, (3) arayüz değişse bile (PyQt → web) motor aynı kalır. Şekil 0'daki katmanlar bu ilkeye göre çizilmiştir.

Tanım

MVC (Model-View-Controller): yaygın bir arayüz mimarisi. **Model** veriyi ve kuralları tutar (simülasyon motoru), **View** ekranı çizer, **Controller** kullanıcının tıklamalarını modele iletir.

9.2 Olay güdümlü programlama

Tanım

Event-driven (olay güdümlü) programlama: programın yukarıdan aşağı akmak yerine olayları (tıklama, zamanlayıcı, yeni log satırı) bekleyip her olaya tepki verdiği yapı. Tüm arayüz kütüphaneleri böyle çalışır.

Örnek: kullanıcı "P1, R2'yi istesin" butonuna tıklar → controller motora *request* çağırısı yapar → motor durumu günceller, tespit ve risk hesaplanır → arayüz yeni durumu çizer.

9.3 Dashboard'da olacaklar

- Özet kartları: aktif process sayısı, kaynak sayısı, anlık risk seviyesi
- Process listesi (durum renkleriyle) ve kaynak listesi (sahibi, kuyruğu)
- RAG çizimi (Hafta 5)
- Deadlock uyarısı ve recovery seçenekleri (Hafta 8)
- Olay akışı (log) ve senaryo yükle / adım adım ilerlet butonları

9.4 PyQt mi, web mi?

Ölçüt	PyQt (masaüstü)	Web (Python sunucu + tarayıcı)
Öğrenilecek dil	Yalnızca Python	Python + HTML/CSS/JavaScript
Kurulum	pip install PyQt6	Flask/FastAPI + ön yüz kütüphaneleri
Graf çizimi	QGraphicsView veya matplotlib	vis-network, Cytoscape.js (çok güçlü)
Görünüm	Klasik masaüstü uygulaması	Modern, esnek tasarım
Demo	Uygulamayı açmak yeterli	Sunucuyu başlatıp tarayıcıda açmak gerekir
Ekip için zorluk	Daha az hareketli parça	İki dünya (sunucu + tarayıcı) arasında iletişim

Projemizde

Karar henüz verilmedi. Ekibin JavaScript bilgisi azsa PyQt daha düz bir yoldur; görsel etkileycilik öncelikliyse web daha çok imkân sunar. Bu kararı 5. haftadan önce birlikte vereceğiz.

Bu bölümün sonunda bilmeniz gerekenler

- Sorumlulukların ayrılması ilkesini ve bize ne kazandırdığını
- MVC'deki üç parçanın görevini
- Olay güdümlü programlamayı
- PyQt ve web seçeneklerinin artı ve eksilerini

10. Hafta — Test ve İyileştirme

Sistemin doğru ve kararlı çalıştığını kanıtlamak

10.1 Test türleri

Test türü	Neyi test eder?	Projemizde örnek
Unit test (birim testi)	Tek bir fonksiyon ya da sınıf	has_cycle, safety fonksiyonları
Integration test (entegrasyon)	Modüllerin birlikte çalışması	Motor + tespit: request sonrası deadlock bulunuyor mu?
System test (sistem testi)	Uygulamanın baştan sona davranışı	Senaryo dosyası yüklenip arayüzde doğru sonuç görülüyor mu?
Regression test (gerileme)	Yeni değişikliğin eski özellikleri bozmadığı	Recovery eklendikten sonra tespit testleri hâlâ geçiyor mu?

10.2 Test senaryosu nasıl yazılır?

Her test üç parçadan oluşur: **hazırlık** (başlangıç durumu), **eylem** (yapılan işlem) ve **beklenen sonuç**. Test, gerçek sonuç beklenenle aynıysa geçer.

No	Senaryo	Beklenen sonuç
1	Normal çalışma: process'ler kaynakları sırayla alıp bırakıyor	Deadlock yok, risk LOW
2	Tek deadlock: P1-R1-P2-R2 çapraz istek	P1 ve P2 deadlock'ta, risk CRITICAL
3	Çoklu deadlock: iki ayrı döngü	İki döngü ayrı ayrı raporlanıyor
4	Recovery: senaryo 2 + kurban sonlandırma	Deadlock kalmadı, sistem SAFE
5	Şekil 3 durumu: döngü var ama deadlock yok	Deadlock yok (en sık yapılan hatayı yakalar)

10.3 pytest ile birim testi

Tanım

pytest, Python'un en yaygın test aracıdır. Adı `test_` ile başlayan fonksiyonları bulup çalıştırır; `assert` satırı yanlışsa testi başarısız sayar.

```
from detection import has_cycle

def test_iki_process_deadlock():
    graph = {"P1": ["R2"], "R2": ["P2"], "P2": ["R1"], "R1": ["P1"]}
    assert has_cycle(graph) is True

def test_dongu_yok():
    graph = {"P1": ["R1"], "R1": []}
    assert has_cycle(graph) is False
```

```
$ pytest
===== 2 passed in 0.01s =====
```

10.4 Uç durumlar (edge cases)

Hataların çoğu "normal" durumlarda değil, uçlarda çıkar. Mutlaka denenmesi gerekenler:

- Hiç process ya da kaynak yokken
- Bir process kendi tuttuğu kaynağı tekrar isterse

- Tutmadığı bir kaynağı bırakmaya çalışırsa
- Sonlandırılmış (TERMINATED) bir process işlem yapmaya çalışırsa
- Kaynağın toplam adedinden fazlası istenirse

Tanım

Debugging (hata ayıklama): hatanın nedenini bulma süreci. Önce hatayı tekrar üretin, sonra log'a bakın, sonra düzeltip aynı hatayı yakalayan bir test ekleyin; böylece hata bir daha geri gelmez.

Bu bölümün sonunda bilmeniz gerekenler

- Dört test türünü ve farklarını
- Bir test senaryosunun üç parçasını
- pytest ile basit bir birim testinin nasıl yazılıp çalıştırıldığını
- Uç durumların neden önemli olduğunu

11. Hafta — Final: Demo, Rapor ve Sunum

Projeyi baştan sona çalışır halde sunmak

11.1 Final demo senaryosu

- Sistemi açıyoruz; üç process ve üç kaynak oluşturuyoruz.
- Kaynakları dağıtıyoruz; risk LOW.
- İki process'in birbirini beklediği senaryoyu kuruyoruz; risk önce MEDIUM, sonra HIGH oluyor.
- Deadlock oluşuyor; risk CRITICAL, döngü grafikte kırmızı.
- Sistem döngüdeki process ve kaynakları gösteriyor, recovery seçeneklerini sunuyor.
- Önerilen kurbanı sonlandırıyoruz; kaynaklar serbest kalıyor, sistem SAFE durumuna dönüyor.

Dikkat

Canlı demoda en sık yaşanan sorun, o an beklenmeyen bir hatadır. Önlem olarak:

- Demo senaryosunu JSON dosyası olarak hazırlayın, elle tıklamaya güvenmeyin.
- En az iki kez prova edin, bir kez de başka bir bilgisayarda.
- Yedek olarak demonun ekran kaydını hazırlayın.

11.2 Proje raporunun bölümleri

Bölüm	İçerik
Özet	Projenin yarım sayfalık özeti
Giriş ve problem tanımı	Deadlock nedir, neden önemli, projenin amacı
Teorik arka plan	Coffman koşulları, RAG, detection, Banker's, recovery (bu kitaptan yararlanılabilir)
Tasarım	Mimari şema, sınıflar, veri yapıları, arayüz tasarımı
Gerçekleştirme	Her modülün nasıl kodlandığı, önemli kod parçaları
Test ve sonuçlar	Test senaryoları, sonuç tablosu, ekran görüntüleri
Sonuç ve gelecek çalışmalar	Neler başarılı, eksikler, gerçek sisteme entegrasyon fikri
Kaynakça	Silberschatz ve kullanılan diğer kaynaklar
Ekip katkıları	Kim hangi modülü yaptı

11.3 Sunum ipuçları

- Her ekip üyesi kendi modülünü anlatsın; hoca herkesin projeyi anladığını görmek ister.
- Teoriyi az, demoyu çok tutun: önce 2-3 slaytla problem, sonra canlı demo.
- Şekil 2'deki gibi basit bir örnekle başlayın; herkes döngüyü görsün.
- Sorulabilecek sorulara hazırlanın: "Döngü her zaman deadlock mıdır?", "Banker's neden pratikte az kullanılır?", "Kurbanı neye göre seçtiniz?".

Bu bölümün sonunda bilmeniz gerekenler

- Demo akışını ve risklerine karşı önlemleri
- Raporun bölümlerini
- Sunumda sorulabilecek temel soruların cevaplarını

Ek A — Yazılım Mühendisliği Pratikleri

Gereksinimler, Git ve ekip çalışması

A.1 Gereksinimler

Tanım

Fonksiyonel gereksinim: sistemin **ne yapacağı**.

Fonksiyonel olmayan gereksinim: sistemin **nasıl** olacağı (hız, kullanılabilirlik, taşınabilirlik).

Tür	Deadlock Guardian'dan örnekler
Fonksiyonel	Sistem process ve kaynak oluşturabilmelidir. · Deadlock'u tespit edip döngüyü göstermelidir. · Güvensiz isteklerde uyarı vermelidir. · En az bir recovery yöntemi uygulamalıdır.
Fonksiyonel olmayan	Tespit sonucu 1 saniyeden kısa sürede gösterilmelidir. · Arayüz ilk kez kullanan biri tarafından anlaşılabilir. · Windows ve macOS'ta çalışmalıdır. · Kod modüler olmalıdır.

A.2 Git ve GitHub

Kavram	Anlamı
Repository (depo)	Projenin tüm dosyalarını ve tüm geçmişini tutan klasör
Commit	Değişikliklerin açıklamalı bir anlık kaydı ("Resource sınıfına kuyruk eklendi")
Branch (dal)	Ana koddan ayrılıp bağımsız çalışılan kopya
Merge (birleştirme)	Bir dalın değişikliklerini başka dala katmak
Pull request (PR)	"Dalımı ana koda katmak istiyorum" talebi; ekip inceler, onaylar
Merge conflict (çakışma)	İki kişi aynı satırı farklı değiştirdiğinde Git'in karar verememesi; elle çözülür
Clone / pull / push	Depoyu indirmek / güncellemeleri almak / kendi commit'lerini göndermek

A.3 Dört kişilik ekip için önerilen çalışma düzeni

- **main** dalı her zaman çalışan kodu tutar; kimse doğrudan main'e yazmaz.
- Herkes kendi işi için dal açar: *feature/simulasyon*, *feature/tespit*, *feature/risk*, *feature/arayuz*.
- Küçük ve sık commit atılır; mesaj ne yapıldığını söyler.
- İş bitince pull request açılır, en az bir ekip arkadaşı inceler (code review), sonra main'e birleştirilir.
- Haftada bir ortak toplantıda dallar birleştirilir ve o haftanın çıktısı birlikte çalıştırılır.
- Her modülün testleri main'e birleştirmeden önce geçmelidir.

A.4 Önerilen klasör yapısı

```
deadlock-guardian/
├── src/
│   ├── simulation/      # Process, Resource, motor          (Üye 1)
│   ├── detection/       # RAG, DFS, detection algoritması   (Üye 2)
│   ├── risk/           # Banker's, risk seviyeleri         (Üye 3)
│   ├── recovery/       # kurban seçimi, kurtarma            (Üye 2)
│   └── ui/             # arayüz ve çizim                   (Üye 4)
├── scenarios/          # JSON senaryo dosyaları
├── tests/              # pytest testleri
├── README.md
└── requirements.txt    # kullanılan Python paketleri
```

Ek B — Python Hızlı Başvuru

Projede en çok kullanacağımız yapılar

Yapı	Örnek	Ne işe yarar?
Liste (list)	held = ["R1", "R2"]	Sıralı eleman topluluğu; append, remove
Sözlük (dict)	alloc = {"P1": [1, 0]}	Anahtar → değer eşlemesi; tablolar için ideal
Küme (set)	visited = set()	Tekrarsız eleman topluluğu; "daha önce gördük mü?"
deque	q = deque(); q.append(p); q.popleft()	Hızlı kuyruk (FIFO)
Enum	class State(Enum): RUNNING = 1	Sabit değer listesi; yazım hatasını önler
Sınıf	class Process: def __init__(self, pid): ...	Nesne kalıbı
f-string	f"{p.pid} bekliyor"	Değişkenleri metne yerleştirme
Liste üretici	[p for p in procs if p.state == WAITING]	Filtrelenmiş yeni liste
any / all	all(x <= y for x, y in zip(a, b))	Vektör karşılaştırması (4.5 ve 6.4)
json	data = json.load(open("senaryo.json"))	Senaryo dosyası okuma

Örnek

Bölüm 4 ve 6'daki "vektör ≤ vektör" karşılaştırmasının Python hali:

```
def kucuk_esit(a, b):
    return all(x <= y for x, y in zip(a, b))

kucuk_esit([1, 0], [2, 1])    # True
kucuk_esit([3, 2], [2, 1])    # False
```

Ek C — Sözlük

Türkçe - İngilizce terimler

Türkçe	İngilizce	Kısa anlam
Atama kenarı	Assignment edge	Kaynaktan process'e ok; kaynak verilmiş
Bekleme	Waiting	Process'in bir kaynağı beklediği durum
Birim testi	Unit test	Tek bir fonksiyonun tek başına test edilmesi
Derinlik öncelikli arama	Depth-first search (DFS)	Grafı derinlemesine gezme algoritması
Döngü	Cycle	Okları takip ederek başlangıç düğümüne dönebilmek
Döngüsel bekleme	Circular wait	Process'lerin kapalı bir zincirde birbirini beklemesi
Geri alınamazlık	No preemption	Kaynağın sahibinden zorla alınamaması
Geri alma (kaynak)	Resource preemption	Kaynağı bir process'ten alıp başkasına vermek
Geri sarma	Rollback	Process'i daha önceki güvenli bir noktaya döndürmek
Görmezden gelme	Ignore / ostrich approach	Deadlock için hiçbir şey yapmamak
Güvenli durum	Safe state	Herkesin bitebileceği bir sıranın bulunduğu durum
Güvenli sıra	Safe sequence	Process'lerin sorunsuz bitebileceği sıra
Güvensiz durum	Unsafe state	Güvenli sıra bulunamayan durum (deadlock riski)
Açlık	Starvation	Bir process'in sırasının hiç gelmemesi
İstek kenarı	Request edge	Process'ten kaynağa ok; kaynak isteniyor
Kaçınma	Avoidance	İstekleri güvenliyse vermek (Banker's)
Kaynak	Resource	Process'in ihtiyaç duyduğu sınırlı şey
Kaynak örneği / adedi	Instance	Bir kaynak tipinin birbirinin aynısı kopyalarından biri
Kaynak tahsis grafi	Resource Allocation Graph (RAG)	Process-kaynak ilişkilerinin yönlü grafi
Kaynak tahsisi	Resource allocation	Kaynakların process'lere dağıtılması
Karşılıklı dışlama	Mutual exclusion	Kaynağı aynı anda tek process'in kullanabilmesi
Kilitlenme	Deadlock	Process'lerin birbirini sonsuza kadar beklemesi
Kurban	Victim	Kurtarmada sonlandırılan / kaynağı alınan process
Kurtarma	Recovery	Deadlock'tan sonra sistemi çalışır hale getirmek
Kuyruk	Queue (FIFO)	İlk gelenin ilk çıktığı veri yapısı
Önleme	Prevention	Dört koşuldan birini tamamen imkânsız kılmak
Process kontrol bloğu	Process Control Block (PCB)	İşletim sisteminin process hakkında tuttuğu kayıt
Sezgisel kural	Heuristic	Garanti vermeyen ama pratikte işe yarayan kural
Süreç	Process	Çalışmakta olan program
Tespit	Detection	Oluşmuş deadlock'u bulmak
Tut ve bekle	Hold and wait	Kaynak tutarken başka kaynak beklemek
Bekleme grafi	Wait-for graph	Yalnızca process'lerden oluşan sade bekleme grafi
Yönlü graf	Directed graph	Kenarları oklu olan graf